JULIA BUSH

Phone: 509 063 472 **Email:** j.elizabethbush@gmail.com **Website:** juliabush.dev
Languages: English(native), Polish(2nd language)

SUMMARY

Self-motivated and hardworking 21-year-old full-stack engineer seeking work opportunities to continually expand my skills and collaborate in a team environment. Despite receiving a **37/45** in the International Baccalaureate (IB), I chose a hands-on learning path over university.

WORK EXPERIENCE & OPEN SOURCE

The Odin Project

An open-source web development curriculum with over 1.7 million users. I contributed to the website to give back to the community that played a key role in my development as a programmer.

- Restored expected cursor behavior on logged-in user avatar icons after upgrading from Tailwind CSS v3 to v4, documenting the change for maintainability, and improved visual feedback for copy-to-clipboard actions by implementing light-theme–dependent styles to enhance perceived interactivity.
- Collaborated on GitHub pull requests using Git-based workflows, including code reviews, iterative revisions, and clear commit history to ensure maintainable and high-quality contributions.

lolnames.gg

Next.js web app for league of legends players to check, generate, and track game names quickly and easily

- Enhanced mobile responsiveness by refining Tailwind CSS utility usage, adjusting spacing, layout and component sizing for streamlined usability on smaller screen sizes.
- Implemented full light/dark theme support using next-themes, integrating system/browser preference detection and theme toggling while updating Tailwind CSS styles for consistent theming across components.

PROJECTS

Asteroids - Python Full Stack App (GitHub)

Backend: Python & Pygame with Websockets, Frontend: Vanilla JS, CSS, HTML5 Canvas, Infrastructure: AWS EC2, Docker, Caddy and Vercel

- Implemented a headless Pygame simulation with an asynchronous 60 FPS game loop, including server-side collision detection, input processing, and explicit game state transitions (running, game over, restart).
- Designed a WebSocket-based client/server protocol for continuous, low-latency, two-way communication, broadcasting a full game state snapshot every 16.7 ms over a persistent WebSocket connection.

Tarot AI - Web App (Github)

Frontend: Next.js(App Router), React + Typescript, Tailwind CSS, Framer motion, Open AI API

- Built an interactive Tarot reading app with real-time AI response streaming and typewriter-style UI updates, reducing perceived latency for a user by incrementally rendering content as it is generated.
- Designed reusable client-side logic, including a custom React typewriter hook and prompt-construction that personalizes readings based on custom drawn Tarot cards.

3D Pokeball Catcher & Pokedex - Full Stack App (Github)

Backend: Node.js + Express, Frontend: React + Vite + Typescript, Three.js , Infrastructure: AWS EC2, Docker, Caddy and Vercel

- Deployed a containerized backend on AWS EC2 with Docker and Caddy as a reverse proxy, and a Vite-powered frontend on Vercel, establishing a production-ready full-stack deployment pipeline.
- Designed a Node.js + Express backend with custom REST endpoints and command-based game logic, managing in-memory game state and PokéAPI integration with caching, and validated cache behavior using Vitest-based unit tests(alternative of Jest).
- Built an interactive Pokédex using React, TypeScript, and Three.js with a fully animated 3D Pokéball and probability-based catch logic tied to Pokémon rarity.

Blog Aggregator - CLI Backend Tool (Github)

In terminal tool/backend: Typescript, Drizzle(ORM), PostgreSQL Database. Users follow and find RSS feeds.

- Designed an extensible, easily maintainable middleware-driven CLI command architecture supporting 10+ commands, including user authentication, feed management, background aggregation, and content browsing, enabling rapid feature expansion with minimal code duplication.
- Implemented a normalized PostgreSQL schema with Drizzle ORM to ensure data integrity (deduplication, cascading deletes, unique constraints), improving reliability, consistency, and security

Static Site Generator - CLI Backend Tool (Github)

In terminal tool/backend written in Python with no external libraries. Converts markdown (.md) files into HTML

- Built a dependency-free static site generator in Python similar in concept to Hugo using a custom Markdown parser and HTML node tree, with a recursive build pipeline that preserves directory structure.
- Applied test-driven development(TDD) with comprehensive unit tests covering Markdown parsing, inline formatting, and HTML rendering, improving correctness and making future feature additions safer and faster.